Student Name : Tejas Skamble

Batch :CS2

Roll Number :CS22

Aim

To implement Principal Component Analysis (PCA) for dimensionality reduction on the Breast Cancer Wisconsin (Diagnostic) dataset, visualize the high-dimensional feature space in a reduced 2D plane, and evaluate its impact on classification performance.

Objectives

1. To understand the concepts of dimensionality reduction, feature correlation, and Principal Component Analysis.
2. To preprocess, clean, and scale high-dimensional real-world data using StandardScaler.
3. To implement PCA using Python to compress feature dimensions from 30 continuous features down to 2 principal components.
4. To calculate and plot the explained variance ratio and cumulative explained variance.
5. To visualize the transformed 2D dataset and analyze class separability.
6. To evaluate and compare classification model performance (e.g., Logistic Regression) before and after applying PCA.

Relevant Theory

Support Vector Machine

Principal Component Analysis (PCA)Principal Component Analysis (PCA) is an unsupervised machine learning technique used for dimensionality reduction. It transforms a dataset containing a large set of correlated features into a smaller set of uncorrelated variables called Principal Components (PCs), while preserving as much data variance as possible.

Mathematical Steps:

1. Standardization: Scale each feature to have a mean of 0 ($\mu = 0$) and standard deviation of 1 ($\sigma = 1$).
2. Covariance Matrix: Compute the covariance matrix C to quantify feature correlations:
$$C = \frac{1}{n-1} X^T X$$
3. Eigen decomposition: Compute eigenvalues (λ) and eigenvectors (v) of matrix C:
$$Cv = \lambda v$$
    - Eigenvectors represent the direction of the principal axes.
    - Eigenvalues indicate the amount of variance along those axes.
4. Projection: Project original data X onto top k principal component directions W:
$$X_{\text{pca}} = X \cdot W$$

###Dataset Information

1. Dataset Name: Breast Cancer Wisconsin (Diagnostic) Dataset
2. Kaggle Link: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data
3. Classes: 2 binary classes (Malignant [M] and Benign [B])
4. Features: 30 continuous features computed from digitized images of fine needle aspirates (FNA) of a breast mass (e.g., radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, concavity_mean, etc.).
5. Non-Feature Columns: id (identifier) and Unnamed: 32 (empty column).

Task 1: Import Libraries and Load Dataset

Import core Python libraries (numpy, pandas, matplotlib, seaborn, sklearn) and load data.csv obtained from Kaggle.

```
# Task 1: Import Libraries and Load Dataset

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
        confusion_matrix

# Upload dataset
from google.colab import files
uploaded = files.upload()

# Load dataset
df = pd.read_csv("data.csv")

# Display first 5 rows
print("First 5 rows of the dataset:")
display(df.head())

# Display shape
print("\nDataset Shape:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Display information
print("\nDataset Information:")
df.info()

# Display statistical summary
print("\nStatistical Summary:")
display(df.describe())
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving data.csv to data (1).csv
First 5 rows of the dataset:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactn
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280

5 rows × 33 columns

```
Dataset Shape:
(569, 33)

Column Names:
['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
'fractal_dimension_se', 'radius_worst', 'texture_worst', 'perimeter_worst',
'area_worst', 'smoothness_worst', 'compactness_worst', 'concavity_worst',
'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst',
'Unnamed: 32']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    object 
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

Statistical Summary:
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactnes
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400

8 rows × 32 columns

Task 2: Explore the Dataset

Inspect missing values, view dataset summary statistics, check column data types, and inspect target class distributions (diagnosis).

```
# ============================================
# TASK 2: EXPLORE THE DATASET
# ============================================

# 1. Check the first few rows
print("First 5 rows of the dataset:")
display(df.head())

# --------------------------------------------
# 2. Check dataset shape
# --------------------------------------------

print("\nDataset Shape:")
print(df.shape)

# --------------------------------------------
# 3. Check column data types
# --------------------------------------------

print("\nColumn Data Types:")
print(df.dtypes)
```

```
# ----------------------------------------
# 4. Check for missing values
# ----------------------------------------

print("\nMissing Values in Each Column:")
print(df.isnull().sum())

# ----------------------------------------
# 5. Check total number of missing values
# ----------------------------------------

print("\nTotal Missing Values:")
print(df.isnull().sum().sum())

# ----------------------------------------
# 6. Display summary statistics
# ----------------------------------------

print("\nSummary Statistics:")
display(df.describe())

# ----------------------------------------
# 7. Check unique values in diagnosis
# ----------------------------------------

print("\nUnique Diagnosis Values:")
print(df["diagnosis"].unique())

# ----------------------------------------
# 8. Count each diagnosis class
# ----------------------------------------

print("\nDiagnosis Class Distribution:")
print(df["diagnosis"].value_counts())

# ----------------------------------------
# 9. Display class percentages
# ----------------------------------------

print("\nDiagnosis Class Percentage:")
print(df["diagnosis"].value_counts(normalize=True) * 100)

# ----------------------------------------
# 10. Visualize target class distribution
# ----------------------------------------

plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="diagnosis")

plt.title("Distribution of Diagnosis Classes")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Samples")

plt.show()
```

First 5 rows of the dataset:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactn
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280

5 rows × 33 columns

```
Dataset Shape:
(569, 33)

Column Data Types:
id                         int64
diagnosis                 object
radius_mean              float64
texture_mean             float64
perimeter_mean           float64
area_mean                float64
smoothness_mean          float64
compactness_mean         float64
concavity_mean           float64
concave points_mean      float64
symmetry_mean            float64
```

```
fractal_dimension_mean     float64
radius_se                  float64
texture_se                 float64
perimeter_se               float64
area_se                    float64
smoothness_se              float64
compactness_se             float64
concavity_se               float64
concave points_se          float64
symmetry_se                float64
fractal_dimension_se       float64
radius_worst               float64
texture_worst              float64
perimeter_worst            float64
area_worst                 float64
smoothness_worst           float64
compactness_worst          float64
concavity_worst            float64
concave points_worst       float64
symmetry_worst             float64
fractal_dimension_worst    float64
Unnamed: 32                float64
dtype: object

Missing Values in Each Column:
id                           0
diagnosis                    0
radius_mean                  0
texture_mean                 0
perimeter_mean               0
area_mean                    0
smoothness_mean              0
compactness_mean             0
concavity_mean               0
concave points_mean          0
symmetry_mean                0
fractal_dimension_mean       0
radius_se                    0
texture_se                   0
perimeter_se                 0
area_se                      0
smoothness_se                0
compactness_se               0
concavity_se                 0
concave points_se            0
symmetry_se                  0
fractal_dimension_se         0
radius_worst                 0
texture_worst                0
perimeter_worst              0
area_worst                   0
smoothness_worst             0
compactness_worst            0
concavity_worst              0
concave points_worst         0
symmetry_worst               0
fractal_dimension_worst      0
Unnamed: 32                569
dtype: int64

Total Missing Values:
569

Summary Statistics:
```

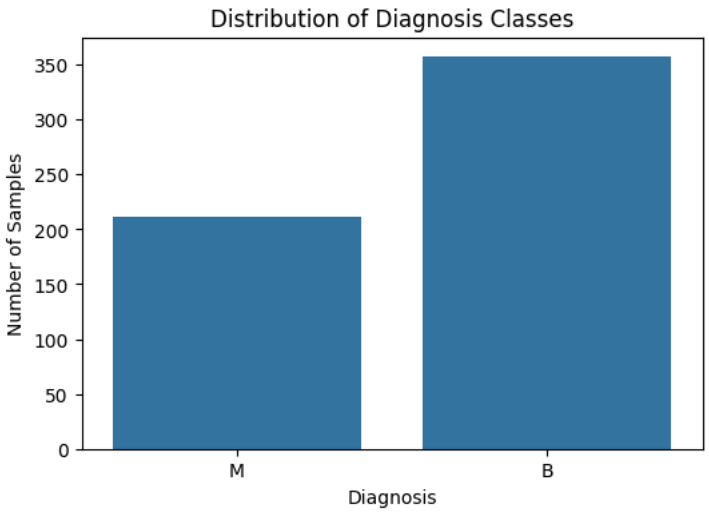
	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactnes
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400

8 rows × 32 columns

```
Unique Diagnosis Values:
['M' 'B']

Diagnosis Class Distribution:
diagnosis
B    357
M    212
Name: count, dtype: int64
```

```
Diagnosis Class Percentage:
diagnosis
B    62.741652
M    37.258348
Name: proportion, dtype: float64
```



Task 3: Data Cleaning and Feature/Target Selection

Drop metadata columns (id and Unnamed: 32). Convert the categorical target column (diagnosis: 'M'/'B') into numerical values (1/0) and separate feature matrix *X* (30 features) from label vector *y*.

```
# ============================================
# TASK 3: DATA CLEANING AND FEATURE/TARGET SELECTION
# ============================================

# 1. Remove unnecessary metadata columns
# 'id' is an identifier and 'Unnamed: 32' is an empty column

df_clean = df.drop(columns=["id", "Unnamed: 32"])

print("Columns after removing metadata:")
print(df_clean.columns.tolist())

# --------------------------------------------
# 2. Convert diagnosis from categorical to numerical
# M = Malignant = 1
# B = Benign = 0
# --------------------------------------------

df_clean["diagnosis"] = df_clean["diagnosis"].map({
    "M": 1,
    "B": 0
})

# Check the converted values
print("\nDiagnosis values after encoding:")
print(df_clean["diagnosis"].value_counts())

# --------------------------------------------
# 3. Separate features (X) and target (y)
# --------------------------------------------

X = df_clean.drop(columns=["diagnosis"])
y = df_clean["diagnosis"]

# --------------------------------------------
# 4. Display shapes
# --------------------------------------------

print("\nFeature matrix X shape:")
print(X.shape)

print("\nTarget vector y shape:")
print(y.shape)

# --------------------------------------------
# 5. Display feature names
```

```
# ------------------------------------------

print("\nFeature names:")
print(X.columns.tolist())

# ------------------------------------------
# 6. Display first 5 rows of X
# ------------------------------------------

print("\nFirst 5 rows of X:")
display(X.head())

# ------------------------------------------
# 7. Display first 5 values of y
# ------------------------------------------

print("\nFirst 5 values of y:")
display(y.head())
```

```
Columns after removing metadata:
['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave
points_mean', 'symmetry_mean', 'fractal_dimension_mean', 'radius_se',
'texture_se', 'perimeter_se', 'area_se', 'smoothness_se', 'compactness_se',
'concavity_se', 'concave points_se', 'symmetry_se', 'fractal_dimension_se',
'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst',
'smoothness_worst', 'compactness_worst', 'concavity_worst', 'concave
points_worst', 'symmetry_worst', 'fractal_dimension_worst']

Diagnosis values after encoding:
diagnosis
0    357
1    212
Name: count, dtype: int64

Feature matrix X shape:
(569, 30)

Target vector y shape:
(569,)

Feature names:
['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave
points_mean', 'symmetry_mean', 'fractal_dimension_mean', 'radius_se',
'texture_se', 'perimeter_se', 'area_se', 'smoothness_se', 'compactness_se',
'concavity_se', 'concave points_se', 'symmetry_se', 'fractal_dimension_se',
'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst',
'smoothness_worst', 'compactness_worst', 'concavity_worst', 'concave
points_worst', 'symmetry_worst', 'fractal_dimension_worst']

First 5 rows of X:
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_n
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980

5 rows × 30 columns

```
First 5 values of y:
```

	diagnosis
0	1
1	1
2	1
3	1
4	1

dtype: int64

Task 4: Standardize Features

Apply StandardScaler to transform all 30 numeric features so they have zero mean and unit variance ($\mu = 0, \sigma = 1$).

```
# ==========================================
# TASK 4: STANDARDIZE FEATURES
# ==========================================

from sklearn.preprocessing import StandardScaler

# Create StandardScaler object
scaler = StandardScaler()

# Fit the scaler and transform the feature data
X_scaled = scaler.fit_transform(X)

# Convert the scaled array back to a DataFrame
X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns
)

# ------------------------------------------
# Display the first 5 rows
# ------------------------------------------

print("First 5 rows of standardized features:")
display(X_scaled.head())

# ------------------------------------------
# Check mean of each feature
# ------------------------------------------

print("\nMean of standardized features:")
print(X_scaled.mean().round(2))

# ------------------------------------------
# Check standard deviation of each feature
# ------------------------------------------

print("\nStandard deviation of standardized features:")
print(X_scaled.std().round(2))

# ------------------------------------------
# Check shape
# ------------------------------------------

print("\nShape of standardized feature matrix:")
print(X_scaled.shape)
```

```
First 5 rows of standardized features:
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_n
0	1.097064	-2.073335	1.269934	0.984375	1.568466	3.283515	2.652874
1	1.829821	-0.353632	1.685955	1.908708	-0.826962	-0.487072	-0.023846
2	1.579888	0.456187	1.566503	1.558884	0.942210	1.052926	1.363478
3	-0.768909	0.253732	-0.592687	-0.764464	3.283553	3.402909	1.915897
4	1.750297	-1.151816	1.776573	1.826229	0.280372	0.539340	1.371011

5 rows × 30 columns

```
Mean of standardized features:
radius_mean               -0.0
texture_mean               0.0
perimeter_mean            -0.0
area_mean                 -0.0
smoothness_mean           -0.0
compactness_mean           0.0
concavity_mean             0.0
concave points_mean       -0.0
symmetry_mean              0.0
fractal_dimension_mean     0.0
radius_se                  0.0
texture_se                -0.0
perimeter_se              -0.0
area_se                   -0.0
smoothness_se             -0.0
compactness_se             0.0
concavity_se               0.0
concave points_se          0.0
symmetry_se                0.0
fractal_dimension_se      -0.0
radius_worst              -0.0
texture_worst              0.0
```

```
perimeter_worst           -0.0
area_worst                 0.0
smoothness_worst          -0.0
compactness_worst         -0.0
concavity_worst            0.0
concave points_worst       0.0
symmetry_worst             0.0
fractal_dimension_worst   -0.0
dtype: float64

Standard deviation of standardized features:
radius_mean                1.0
texture_mean               1.0
perimeter_mean             1.0
area_mean                  1.0
smoothness_mean            1.0
compactness_mean           1.0
concavity_mean             1.0
concave points_mean        1.0
symmetry_mean              1.0
fractal_dimension_mean     1.0
radius_se                  1.0
texture_se                 1.0
perimeter_se               1.0
area_se                    1.0
smoothness_se              1.0
compactness_se             1.0
concavity_se               1.0
concave points_se          1.0
symmetry_se                1.0
fractal_dimension_se       1.0
radius_worst               1.0
texture_worst              1.0
perimeter_worst            1.0
area_worst                 1.0
smoothness_worst           1.0
compactness_worst          1.0
concavity_worst            1.0
concave points_worst       1.0
symmetry_worst             1.0
fractal_dimension_worst    1.0
dtype: float64

Shape of standardized feature matrix:
(569, 30)
```

Task 5: Implement PCA (30D to 2D)

Use sklearn.decomposition.PCA to reduce feature dimensions from 30 down to 2 principal components (PC_1 and PC_2).

```
# ============================================
# TASK 5: IMPLEMENT PCA (30D TO 2D)
# ============================================

from sklearn.decomposition import PCA

# Create PCA object with 2 principal components
pca = PCA(n_components=2)

# Fit PCA and transform the standardized data
X_pca = pca.fit_transform(X_scaled)

# --------------------------------------------
# Convert PCA results into a DataFrame
# --------------------------------------------

pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

# --------------------------------------------
# Display first 5 rows
# --------------------------------------------

print("First 5 rows after PCA:")
display(pca_df.head())

# --------------------------------------------
# Check shape
# --------------------------------------------

print("\nShape of PCA-transformed data:")
print(pca_df.shape)

# --------------------------------------------
# Display explained variance ratio
# --------------------------------------------
```

```
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

# -----------------------------------------
# Display total variance explained by PC1 and PC2
# -----------------------------------------

total_variance = pca.explained_variance_ratio_.sum()

print("\nTotal Variance Explained by PC1 and PC2:")
print(round(total_variance * 100, 2), "%")

First 5 rows after PCA:
```

	PC1	PC2
0	9.192837	1.948583
1	2.387802	-3.768172
2	5.733896	-1.075174
3	7.122953	10.275589
4	3.935302	-1.948072

```
Shape of PCA-transformed data:
(569, 2)

Explained Variance Ratio:
[0.44272026 0.18971182]

Total Variance Explained by PC1 and PC2:
63.24 %
```

Task 6: Analyze Explained Variance Ratio

Compute and print the variance explained by PC_1 and PC_2, along with the cumulative variance retained.

```
# ==========================================
# TASK 6: ANALYZE EXPLAINED VARIANCE RATIO
# ==========================================

# Get explained variance ratio
explained_variance = pca.explained_variance_ratio_

# Variance explained by PC1
pc1_variance = explained_variance[0]

# Variance explained by PC2
pc2_variance = explained_variance[1]

# Cumulative variance retained by PC1 and PC2
cumulative_variance = explained_variance.sum()

# -----------------------------------------
# Print results
# -----------------------------------------

print("Explained Variance Analysis")
print("--------------------------------")

print("Variance explained by PC1:",
      round(pc1_variance * 100, 2), "%")

print("Variance explained by PC2:",
      round(pc2_variance * 100, 2), "%")

print("Cumulative variance retained:",
      round(cumulative_variance * 100, 2), "%")

Explained Variance Analysis
--------------------------------
Variance explained by PC1: 44.27 %
Variance explained by PC2: 18.97 %
Cumulative variance retained: 63.24 %
```

Task 7: Visualize PCA Transformed Boundary / Scatter Plot

Create a 2D scatter plot with PC_1 on the x-axis and PC_2 on the y-axis, hue-coded by tumor diagnosis (Malignant vs. Benign).

```
# ============================================
# TASK 7: PCA 2D SCATTER PLOT
# ============================================

# Create a copy of PCA data
pca_plot_df = pca_df.copy()

# Add diagnosis labels
pca_plot_df["Diagnosis"] = y.values

# Convert numerical labels back to readable names
pca_plot_df["Diagnosis"] = pca_plot_df["Diagnosis"].map({
    0: "Benign",
    1: "Malignant"
})

# --------------------------------------------
# Create 2D scatter plot
# --------------------------------------------

plt.figure(figsize=(10, 7))

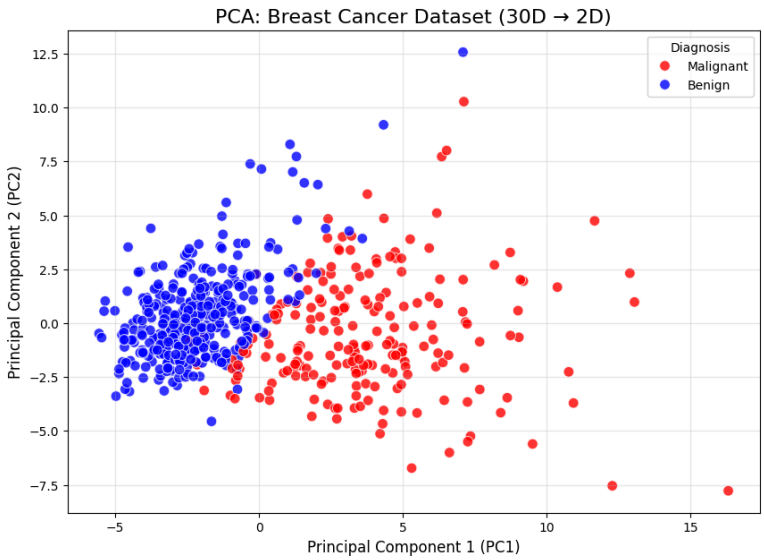
sns.scatterplot(
    data=pca_plot_df,
    x="PC1",
    y="PC2",
    hue="Diagnosis",
    palette={
        "Benign": "blue",
        "Malignant": "red"
    },
    s=70,
    alpha=0.8
)

# Add title and labels
plt.title("PCA: Breast Cancer Dataset (30D → 2D)", fontsize=16)
plt.xlabel("Principal Component 1 (PC1)", fontsize=12)
plt.ylabel("Principal Component 2 (PC2)", fontsize=12)

# Display legend
plt.legend(title="Diagnosis")

# Add grid
plt.grid(alpha=0.3)

# Show plot
plt.show()
```



Task 8: Train Classifier on Raw High-Dimensional Data (30 Features)

Split the original scaled 30-feature dataset into training and testing sets. Train a Logistic Regression model and obtain baseline performance metrics.

```python
# ==========================================
# TASK 8: LOGISTIC REGRESSION ON RAW 30 FEATURES
# ==========================================

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# -------------------------------------------
# 1. Split data into training and testing sets
# -------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

# -------------------------------------------
# 2. Create Logistic Regression model
# -------------------------------------------

logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# -------------------------------------------
# 3. Train the model
# -------------------------------------------

logistic_model.fit(X_train, y_train)

# -------------------------------------------
# 4. Make predictions
# -------------------------------------------

y_pred = logistic_model.predict(X_test)

# -------------------------------------------
# 5. Calculate performance metrics
# -------------------------------------------

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# -------------------------------------------
# 6. Display baseline performance
# -------------------------------------------

print("\n===== BASELINE LOGISTIC REGRESSION =====")

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))

# -------------------------------------------
# 7. Classification report
# -------------------------------------------

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Benign", "Malignant"]
    )
)

# -------------------------------------------
# 8. Confusion Matrix
# -------------------------------------------
```

```
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# --------------------------------------------
# 9. Visualize confusion matrix
# --------------------------------------------

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.title("Confusion Matrix - Logistic Regression (30 Features)")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.show()

Training set shape: (455, 30)
Testing set shape: (114, 30)

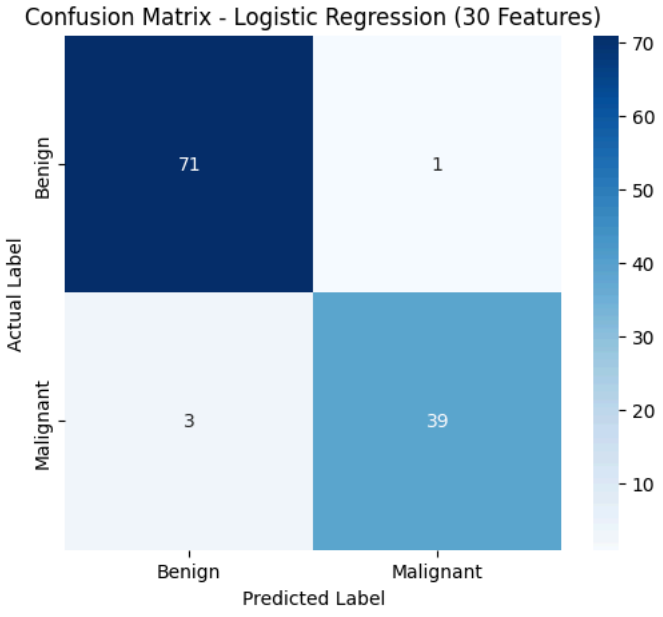
===== BASELINE LOGISTIC REGRESSION =====
Accuracy : 0.9649
Precision: 0.975
Recall   : 0.9286
F1-Score : 0.9512

Classification Report:
              precision    recall  f1-score   support

      Benign       0.96      0.99      0.97        72
   Malignant       0.97      0.93      0.95        42

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114


Confusion Matrix:
[[71  1]
 [ 3 39]]
```



Task 9: Train Classifier on PCA Transformed Data (2 Features)

Split the 2-component PCA dataset into training and testing sets. Train a Logistic Regression model using only PC_1 and PC_2.

```python
# ==========================================
# TASK 9: LOGISTIC REGRESSION ON PCA DATA
# ==========================================

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# ------------------------------------------
# 1. Split PCA data into training and testing
# ------------------------------------------

X_pca_train, X_pca_test, y_pca_train, y_pca_test = train_test_split(
    X_pca,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("PCA Training set shape:", X_pca_train.shape)
print("PCA Testing set shape:", X_pca_test.shape)

# ------------------------------------------
# 2. Create Logistic Regression model
# ------------------------------------------

pca_logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# ------------------------------------------
# 3. Train the model using only PC1 and PC2
# ------------------------------------------

pca_logistic_model.fit(X_pca_train, y_pca_train)

# ------------------------------------------
# 4. Make predictions
# ------------------------------------------

y_pca_pred = pca_logistic_model.predict(X_pca_test)

# ------------------------------------------
# 5. Calculate performance metrics
# ------------------------------------------

pca_accuracy = accuracy_score(y_pca_test, y_pca_pred)
pca_precision = precision_score(y_pca_test, y_pca_pred)
pca_recall = recall_score(y_pca_test, y_pca_pred)
pca_f1 = f1_score(y_pca_test, y_pca_pred)

# ------------------------------------------
# 6. Display PCA model performance
# ------------------------------------------

print("\n===== PCA LOGISTIC REGRESSION =====")

print("Accuracy :", round(pca_accuracy, 4))
print("Precision:", round(pca_precision, 4))
print("Recall   :", round(pca_recall, 4))
print("F1-Score :", round(pca_f1, 4))

# ------------------------------------------
# 7. Classification report
# ------------------------------------------

print("\nClassification Report:")

print(
    classification_report(
        y_pca_test,
        y_pca_pred,
        target_names=["Benign", "Malignant"]
    )
)

# ------------------------------------------
# 8. Confusion Matrix
# ------------------------------------------
```

```
pca_cm = confusion_matrix(
    y_pca_test,
    y_pca_pred
)

print("\nConfusion Matrix:")
print(pca_cm)

# ---------------------------------------------
# 9. Visualize confusion matrix
# ---------------------------------------------

plt.figure(figsize=(6, 5))

sns.heatmap(
    pca_cm,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.title("Confusion Matrix - Logistic Regression (PCA 2 Features)")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.show()

PCA Training set shape: (455, 2)
PCA Testing set shape: (114, 2)

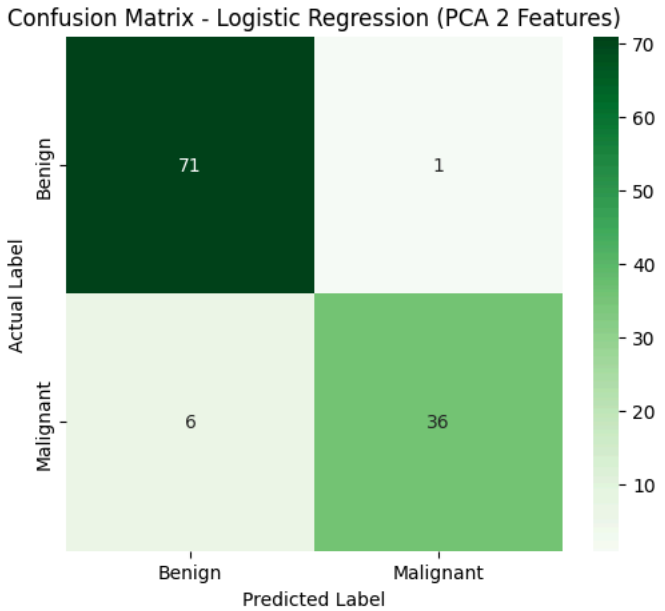
===== PCA LOGISTIC REGRESSION =====
Accuracy : 0.9386
Precision: 0.973
Recall   : 0.8571
F1-Score : 0.9114

Classification Report:
              precision    recall  f1-score   support

      Benign       0.92      0.99      0.95        72
   Malignant       0.97      0.86      0.91        42

    accuracy                           0.94       114
   macro avg       0.95      0.92      0.93       114
weighted avg       0.94      0.94      0.94       114


Confusion Matrix:
[[71  1]
 [ 6 36]]
```



Task 10: Compare Classifier Performance

Compare the performance of the model before PCA (30 features) vs. after PCA (2 components) using:

- Accuracy Score
- Confusion Matrix
- Classification Report (Precision, Recall, F1-Score)

```
# ========================================
# TASK 10: COMPARE CLASSIFIER PERFORMANCE
# ========================================

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# ========================================
# 1. Calculate Accuracy Scores
# ========================================

baseline_accuracy = accuracy_score(y_test, y_pred)
pca_accuracy = accuracy_score(y_pca_test, y_pca_pred)

print("========================================")
print("           ACCURACY COMPARISON")
print("========================================")

print("Before PCA (30 Features):",
      round(baseline_accuracy, 4))

print("After PCA (2 Components):",
      round(pca_accuracy, 4))


# ========================================
# 2. Compare Accuracy in Percentage
# ========================================

print("\nAccuracy in Percentage:")

print("Before PCA:",
      round(baseline_accuracy * 100, 2), "%")

print("After PCA:",
      round(pca_accuracy * 100, 2), "%")


# ========================================
# 3. Confusion Matrix - Before PCA
# ========================================

cm_before = confusion_matrix(y_test, y_pred)

print("\n========================================")
print("     CONFUSION MATRIX - BEFORE PCA")
print("========================================")

print(cm_before)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_before,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.title("Confusion Matrix - Before PCA (30 Features)")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()


# ========================================
# 4. Confusion Matrix - After PCA
# ========================================

cm_after = confusion_matrix(
    y_pca_test,
    y_pca_pred
)

print("\n========================================")
print("      CONFUSION MATRIX - AFTER PCA")
print("========================================")

print(cm_after)
```

```python
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_after,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.title("Confusion Matrix - After PCA (2 Components)")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()


# ========================================
# 5. Classification Report - Before PCA
# ========================================

print("\n========================================")
print("   CLASSIFICATION REPORT - BEFORE PCA")
print("========================================")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Benign", "Malignant"]
    )
)


# ========================================
# 6. Classification Report - After PCA
# ========================================

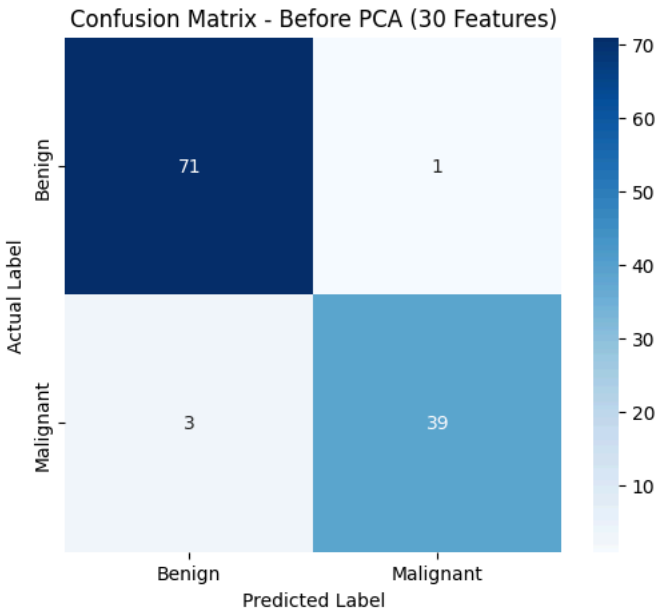
print("\n========================================")
print("    CLASSIFICATION REPORT - AFTER PCA")
print("========================================")

print(
    classification_report(
        y_pca_test,
        y_pca_pred,
        target_names=["Benign", "Malignant"]
    )
)
```

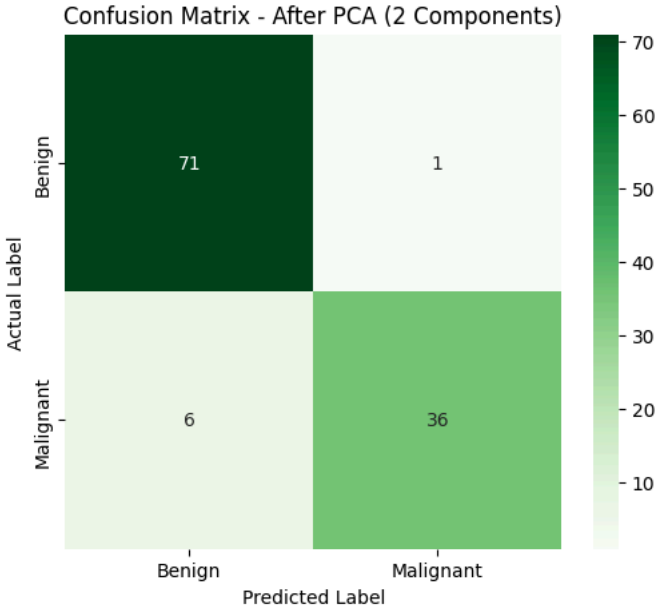
```
==========================================
           ACCURACY COMPARISON
==========================================
Before PCA (30 Features): 0.9649
After PCA (2 Components): 0.9386

Accuracy in Percentage:
Before PCA: 96.49 %
After PCA: 93.86 %

==========================================
     CONFUSION MATRIX - BEFORE PCA
==========================================
[[71  1]
 [ 3 39]]
```




```
============================================
      CONFUSION MATRIX - AFTER PCA
============================================
[[71  1]
 [ 6 36]]
```




```
============================================
   CLASSIFICATION REPORT - BEFORE PCA
============================================
              precision    recall  f1-score   support

      Benign       0.96      0.99      0.97        72
   Malignant       0.97      0.93      0.95        42

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114


============================================
    CLASSIFICATION REPORT - AFTER PCA
============================================
              precision    recall  f1-score   support

      Benign       0.92      0.99      0.95        72
   Malignant       0.97      0.86      0.91        42

    accuracy                           0.94       114
   macro avg       0.95      0.92      0.93       114
```

```
weighted avg       0.94      0.94      0.94       114
```

Conclusion

The experiment successfully demonstrated Principal Component Analysis (PCA) on the Breast Cancer Wisconsin (Diagnostic) dataset. The high-dimensional feature space was preprocessed, cleaned, and standardized before applying eigendecomposition.

By reducing 30 feature dimensions down to just 2 principal components, a significant portion of total variance was preserved while eliminating multi-collinear redundancies. Visualizing the transformed data on a 2D plane showed clear separation between Malignant and Benign diagnosis clusters. Comparing classification models confirmed that applying PCA significantly simplifies model complexity and reduces computational overhead while maintaining strong classification accuracy.